Principles & Practices Document — Leave Management System

Team: Kamalasaravanan, Harish | Repository: github.com/ee24s063/Leave_Management_System | October 2026

Project. A web application for employee leave: login with three roles (Employee, Team Lead, HR), two-level approval, half-day leave, compensatory leave, an auditable balance ledger, an HR dashboard and Excel exports. Stack: Java 17+, Spring Boot 3.1, Spring Security with JWT, SQLite with Flyway migrations, Thymeleaf and Bootstrap. It was built in planned phases with an AI coding assistant (Claude Code); the practices below are how we keep that work correct, traceable and reviewable.

Lifecycle at a glance

1 Plan Written phase plan; design decisions recorded once	→	**2 Prompt** Template: context, task, constraints, done-when	→	**3 Build** Assistant implements one phase; we review the diff	→	**4 Test** Unit, integration and UI tests; screenshots reviewed	→	**5 Debug** Reproduce, evidence, root cause, smallest fix, test	→	**6 Document** README per phase; plan and test report updated	→	**7 Commit** Green suite; code and prompt log in one commit

Steps 4–7 repeat for every phase; a phase is finished only when its tests pass, its README section is current and it is committed.

1. Prompt Engineering

We treat a prompt as a small specification. The principles we apply (sample prompts in the Appendix):

- **Context before the task.** Work starts from a written implementation plan (phases, data model, rules). Prompts refer to it ("continue with phase 4") instead of re-explaining the system, and a fixed opening prompt ("continue the LMS project") makes the assistant report status and open decisions before it writes any code.
- **Decide once, write it down.** Design answers (no lower limit on balances, comp-off never expires, field name `reporting_LeadId`, SQLite stays) are recorded in the plan so later prompts do not reopen them.
- **One scoped task per prompt, in order.** One phase or one change at a time ("complete phase 5 first"), so each result can be tested and reviewed on its own.
- **Ask before deciding.** For risky changes we first ask for trade-offs ("if I switch to JWT login, what will be the issue?"), then give the instruction together with its constraints ("for the demo, localhost is enough").
- **State acceptance criteria and a review gate.** Prompts say what "done" means and when to stop: "generate JUnit tests for each module, run them, conduct UI tests, and share the results in a document before committing."
- **Verify, never trust.** Every generated change must be backed by a test run, a run of the real app, or a screenshot; when an answer misses the point we correct it explicitly instead of re-asking vaguely.

2. Debugging

We follow the same five steps for every issue: **(1) reproduce** it reliably, at the lowest layer possible (HTTP calls with curl before the browser); **(2) collect evidence**: Spring Security DEBUG logs, response headers, test reports, browser screenshots; **(3) find the root cause** and decide whether it is in the app or in the test; **(4) apply the smallest fix**; **(5) add or extend a test**, then rerun the whole suite. Examples from this project:

Symptom	Evidence that located it	Root cause → fix
After the JWT switch, form posts failed with 403 in the real app (tests passed)	Security DEBUG log: "Invalid CSRF token"; a new CSRF cookie on every response	Each token request looked like a new login, so the CSRF token rotated on every page (breaking a second tab) → store the authentication for the request; verified with a two-tab check
Column headers missing in three tables	UI-test screenshot	Global `th` rule (white text) under Bootstrap's white table background → dark header class, as on all other tables
HR list sometimes showed the wrong tab's data	UI test failure + screenshot of a re-drawn table	Late response of the previous tab overwrote the list → ignore answers for a tab the user has left
"Login as another user fails" (suspected)	Same flow reproduced with curl: the server switched users correctly	Test bug, not app bug: the script clicked the navbar's Logout → fixed the test selector; app unchanged

3. Version Control (Git) for Code and Prompts

- **Repository on GitHub**, branch `main`. Commits are made only after the full test suite is green and the user has reviewed the results; messages describe the scope (features, tests, migrations).
- **What is versioned:** source, tests, `pom.xml`, README, and the database schema as ordered Flyway migrations (V1 initial schema → V4 token version), so any checkout rebuilds the same schema. **What is not:** `.gitignore` excludes the SQLite file (real employee data and password hashes) and build output.
- **Prompts are versioned next to the code** in `docs/prompts/PROMPT_LOG.md`: each prompt as written, the principle it applies, its outcome and the commit that holds the result, plus the reusable prompt template and the recorded design decisions. A prompt and the code it produced land in the same commit, so `git log` answers both "what changed" and "what we asked for".
- **History is never rewritten:** GitHub's initial commit was merged, not overwritten. As two of us now work in parallel, each new feature gets its own branch, merged through a pull request the other member reviews.

4. Testing, Refactoring & Documentation

Testing follows a test pyramid; all 245 tests pass in about 60 seconds with `mvn test`, and every bug fix gets a test that would have caught it.

Level	Tools	What it checks	Tests
Unit	JUnit 5, Mockito, AssertJ	Each module alone: utilities, entity rules, JWT/security, every service (balances, approvals, comp-off, employees, reports, Excel)	141
Integration	Spring Boot Test, MockMvc	The whole app through its REST API with its own temporary SQLite database: access rules, workflows, ledger, reports, exports (files opened and checked)	91
UI	Selenium, headless Chrome/Edge	13 end-to-end scenarios clicked as Employee, Team Lead and HR; a screenshot is saved per scenario and reviewed	13

- **Refactoring** is done in small steps behind the tests: one shared approval component serves both leave and comp-off; one generic Excel writer serves all five exports; report totals moved into database queries; the session-based account filter was replaced by a single JWT filter; incorrect helper code found by unit tests was fixed. A change is accepted only when the full suite is still green.
- **Documentation:** the README is updated at the end of every phase (features, roles, setup, REST API tables, settings, known limitations, troubleshooting); the implementation plan and a test report are kept as shared documents; classes carry Javadoc explaining *why*, and each migration explains its change.

Appendix — Sample Prompts

Real prompts from the project (as typed), the principle each one shows, and what it produced. The full log is in `docs/prompts/PROMPT_LOG.md`.

#	Prompt (as written)	Principle	Result
1	continue the LMS project	Recall context; status before code	Status, open decisions and numbered next steps; no code changed
2	stay with sql lite, don't keep the current session login. continue with phase 4	Explicit decisions + one scoped task	HR dashboard and Team Lead view, 11 tests, README updated
3	if i switch to JWT login...what will be the issue?	Ask for trade-offs before deciding	Risks listed (page loads, revocation, logout, CSRF, downloads) and a recommendation
4	ok, switch to JWT. But for demo we local host is enough. So, switch to JWT if required so that we dont need to do that in future.	Instruction with scope and constraints	JWT in an HttpOnly cookie plus Bearer API login, token revocation, 6 tests
5	complete phase 5 first	Prioritise; one phase at a time	Five Excel exports, 10 tests
6	before commiting generate Junit test cases for each modules and run those test cases. Also conduct UI test. Share me the test results in a document to me first.	Acceptance criteria + review gate before commit	154 new tests, 3 defects found and fixed, test report
7	no iam not asking about the test databases. I asking the about the details we give in the leave management system app...	Correct a missed answer explicitly	Precise answer about where app data is stored

Template we now use for new work, applied to the next feature in our schedule:

```
Context:    Leave Management System, Phases 1-5 done (see README and plan).
            Next item in the schedule: contract/tenure tracking.
Task:       Add contract start/end dates, resignation and relieving dates to Employee.
            Show tenure on the Employees page; block leave after the relieving date.
Constraints: Keep SQLite; new Flyway migration (V5); keep the field name reporting_LeadId;
            HR-only editing, as for other employee fields.
Done when: Unit + integration tests cover the new rules, a UI scenario covers the form,
            the full suite is green, and the README is updated.
Output:     Code and tests; show me the test results before committing.
```

Why this template works: the context line saves re-explaining the system; the task is a single change; constraints protect earlier decisions; “done when” turns the prompt into a checkable specification; and the output line keeps a human review gate before anything is committed.